Distributed Locomotion of a Meta-Module Chain with 3D Catoms

Spatial pipelining in VisibleSim

Muhammad Omais Rana

Abstract

A chain of four meta-modules (48 catoms) has to "walk" in one direction: the tail meta-module is taken apart, its catoms travel over the structure, and it is rebuilt at the head of the chain. This report describes a *fully distributed* solution where no catom is in charge. A single *token* travels from catom to catom and decides who moves next. A catom hands the token on as soon as it has moved a few steps away from its source, so two or more catoms are on the way at the same time, like a conveyor belt. When the grid border is reached, the chain simply turns around, so the motion never ends.

1 The problem and the setting

Task. A set of 3D catoms is arranged in a chain of *meta-modules*. Starting from a linear configuration, the left-most meta-module is dismantled, its catoms are transported through the structure, and the meta-module is reassembled on the right side, next to the last one. Repeating this moves the whole chain forward.

Configuration. The configuration file contains 48 catoms forming four meta-modules of 12 catoms each. The shapes alternate between the two mirror-like forms *FrontBack* (FB) and *Back-Front* (BF), so that neighbouring meta-modules interlock. Each meta-module is described by its *root* catom (the cell $(0, 0, 0)$ of the shape) and 12 relative positions.

Meta-module	Colour	Shape	Root cell
1 (tail)	green	FB	$(4, 4, 2)$
2	white	BF	$(8, 4, 2)$
3	orange	FB	$(12, 4, 2)$
4 (head)	pink	BF	$(16, 4, 2)$
new slot	green (after the move)	FB	$(20, 4, 2)$

Table 1: The initial chain. Roots are 4 cells apart along x. The grid is $60 \times 40 \times 40$.

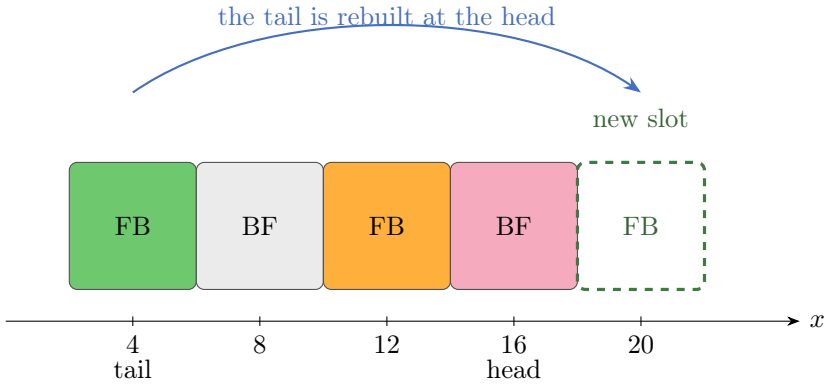


Figure 1: Side view along the x axis. The tail meta-module (green) is dismantled and rebuilt in the free slot next to the head (pink). The shape of the new meta-module is the opposite of the head's shape.

Order of the 12 positions. The exercise gives the 12 relative positions of FB and BF *in a special order*. Removing (and placing) the catoms in this order avoids blockages. The code keeps this order and uses it as the sequence index $0, \ldots, 11$ of a cycle.

2 The idea: a conveyor belt of catoms

Moving 12 catoms *one after the other* works, but it is slow: most of the time only one catom moves. Starting all 12 at once does not work either, because a catom needs its neighbours as pivots when it leaves, and a target cell can only be filled once a neighbouring cell has been filled.

The solution is a rule that every catom can apply with local information only:

> *"The next catom may start as soon as I have moved about six steps away from my source."*

The catom that is moving holds a **token**. After six rotations, the departure zone is free, so the token is handed to the next catom in the sequence. Catoms therefore start one after the other with a small *gap*, like items on a conveyor belt (Figure 2). No catom needs to know how many others are moving.

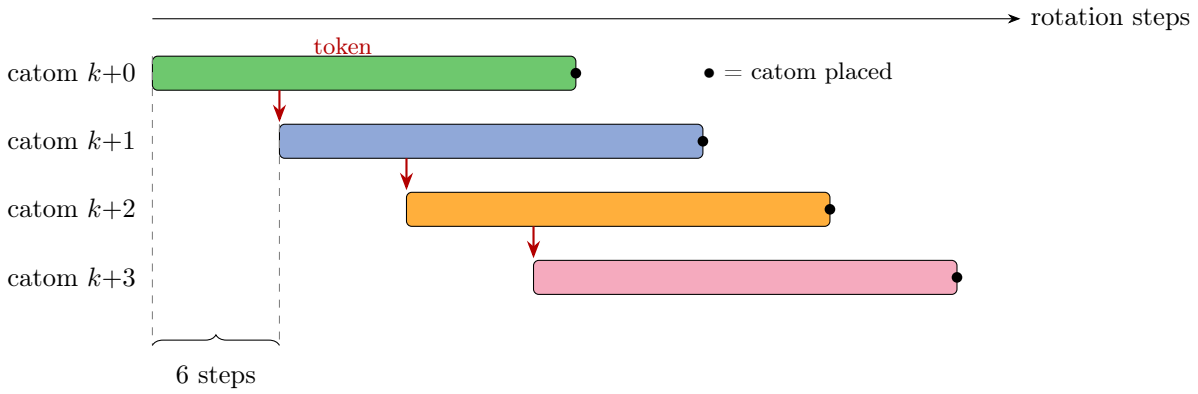


Figure 2: Spatial pipelining (ideal case). Each catom passes the token (red arrow) to the next one after six rotations, so several catoms travel at the same time, with no central controller. A catom whose target is not ready yet waits at its source and keeps the token, which delays the ones behind it.

Why does the pipeline match the shape? The target cells of a meta-module depend on each other: a cell can only be filled once a neighbouring cell is filled. For the FB shape, the neighbour analysis gives the following picture.

Sequence index k	Needs first
0, 1, 2	the previous meta-module (they touch it)
3	index 0
4	index 2
5, 6, 7, 8, 9, 10	index $k-2$
11	index 9 or 10

Table 2: Dependencies between the cells of the shape (ring neighbours). After the first three, each cell needs the one two places before it, so there are two independent chains: $3 \to 5 \to 7 \to 9$ and $4 \to 6 \to 8 \to 10$.

So two catoms can always work in parallel. A third one can only start usefully if its target is already supported. Otherwise it waits at its source (retrying every 0.1–0.3 s) and, because it still holds the token, the catoms behind it wait as well. The pipeline does not need to know the dependencies: when a target is not ready, the catom simply waits, which acts as a natural brake and keeps the order of the sequence.

3 How it works, step by step

3.1 Who starts a cycle?

At start-up, *every* catom runs the same test (`checkInitiator`):

1. Am I the root of a complete meta-module (all 12 cells occupied, FB or BF)?
2. Is there a complete meta-module of the other shape *behind* me (4 cells back)?

Only the root of the **tail** meta-module answers yes to (1) and no to (2). It becomes the *initiator*. It walks along the chain to find the head and computes the destination:

$$\text{newRoot} = \text{headRoot} + 4 \cdot \text{direction}, \qquad \text{newShape} = \text{opposite}(\text{headShape}).$$

```
MMShape s;
if (!detectRoot(module->position, s)) return;   // not a meta-module root
Cell3DPosition back(module->position[0] - dir * MM_SPACING_X,
                    module->position[1], module->position[2]);
if (isComplete(back, opposite(s))) return;      // someone is behind me
```

3.2 The token

The token is a small message that contains everything a catom needs to know:

Field	Meaning
`cycleSeq`	number of the current cycle
`curIdx`	which of the 12 catoms is next (0 to 11)
`movingRight`	direction of the chain
`srcRef`, `srcShape`	root and shape of the meta-module being dismantled
`newRef`, `newShape`	root and shape of the new meta-module
`targetPos`	cell of the catom that must move now
`goalPos`	cell where that catom has to go

Table 3: The `MoveToken` message. Because it carries all this information, any catom can build the next token.

The token is *flooded*: every catom forwards it once to its neighbours (duplicates are ignored using the pair `(cycleSeq, curIdx)`). Only the catom that stands on `targetPos` and is not already moving acts on it.

3.3 Moving one catom

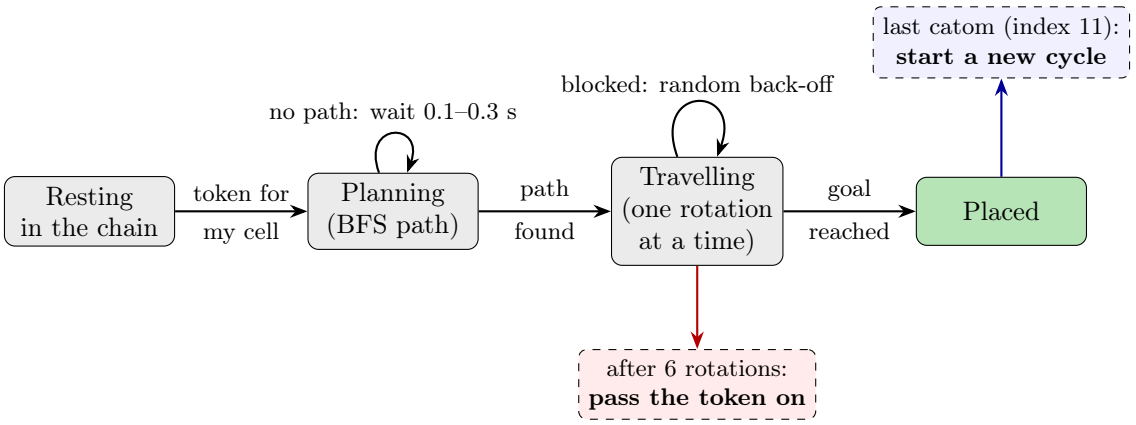


Figure 3: Life of a catom that receives the token.

- **Path.** The catom computes a path to its goal with a breadth-first search (the exercise helper `findPath`). While the goal cannot be reached yet (no neighbour supports it), it waits and tries again.
- **Steps.** The path is followed with one `Catoms3DRotationStartEvent` per step. Before each step, the catom checks that the next cell is free, not reserved by another catom, and that `canMoveTo` holds.
- **Random delays.** A random delay before the start and a random back-off when a cell is blocked keep two catoms from moving into the same cell at the same moment.
- **Reservation.** A cell that a catom is about to enter is reserved in a table shared by all catoms.

```
// in onMotionEnd(): the departure zone is free -> invite the next catom
if (!tokenPassed &&
    (stepIdx >= SAFE_DEPARTURE_STEPS || stepIdx >= path.size() - 1)) {
    passTokenToNext();
}
```

3.4 End of a cycle

When the last catom (index 11) has been placed, it broadcasts a `CycleReset` message with the next cycle number. Every catom clears its list of seen tokens and runs the initiator test again. The root of the new tail (the white meta-module) now answers yes, and the next cycle starts. In this way the starting role simply *moves along the chain*, and no catom keeps a special role.

3.5 Turning around at the grid border

Before starting, the initiator checks that all 12 cells of the new meta-module are inside the grid. If not, it flips the direction, broadcasts a `CycleReset` with the new direction, and the catoms run the initiator test again. Now the root at the *other* end of the chain is the tail, and the meta-modules are dismantled in reverse sequence order (index $11 - k$), so the chain walks back. The motion never ends.

Parameter	Value	Meaning
`SAFE_DEPARTURE_STEPS`	6	rotations before the token is passed on
`START_JITTER_MAX`	5 ms	random delay before a catom starts
`BACKOFF_MIN/MAX`	2–8 ms	wait when the next cell is blocked
`MAX_BACKOFFS`	5	back-offs before a new plan is made
`LOCAL_RETRY_MIN/MAX`	0.1–0.3 s	wait when there is no path yet
`MAX_BFS_NODES`	4000	limit of the path search

Table 4: Main parameters (in the code, times are in microseconds).

4 Running it and demonstration

The program is started with the configuration file:

```
MMLocomotion.exe -c config.xml
```

The console prints one line per event, for example:

```
Cycle <n> Initiator: Dismantling <old root> -> <new root>
catom <id> starting sequence <k>
catom <id> successfully assembled at <cell>
Grid border reached! Reversing chain direction.
```

4.1 Video Demonstration

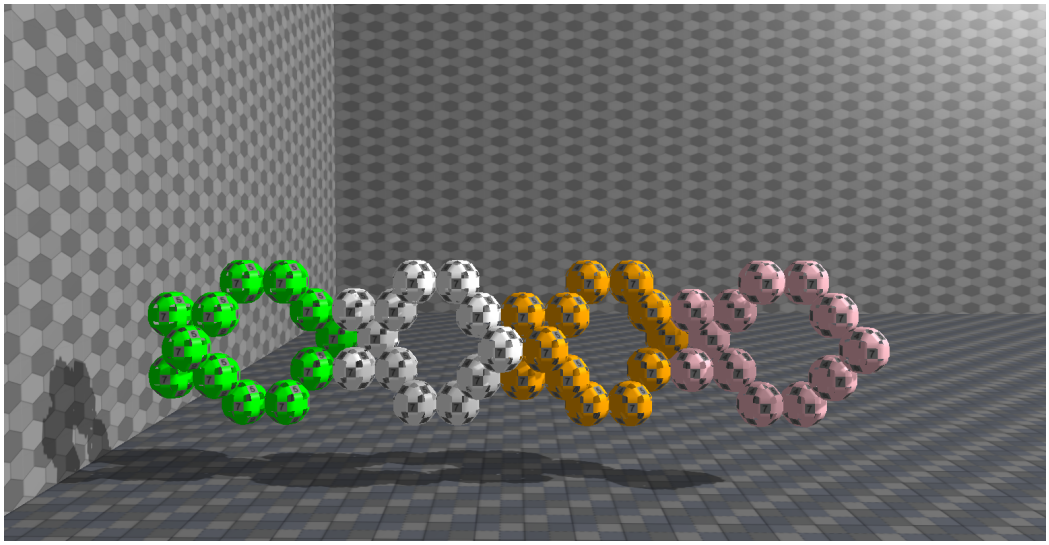


Figure 4: A video file named demo.mp4 has been given in this folder to server as the demonstration.

5 Comparison with the earlier centralised version

	Centralised (coordinator)	Distributed (this report)
Who decides?	one coordinator catom	nobody; a token moves along the chain
Communication	direct calls to the catoms	messages: token and reset
How much moves per cycle	the whole chain shifts by one chain length	only the tail meta-module moves (as in the exercise)
Several catoms at once	window of k commands	emerges from the six-step gap
Failures	watchdog, retries, give-up	none (a stuck catom keeps retrying)

Table 5: The distributed version is simpler and has no single point of failure, but it has no safety net.

6 Conclusion

A chain of meta-modules can walk without any coordinator. The tail meta-module is rebuilt at the head, one catom at a time, and the token that selects the next catom moves along with them. The rule "pass the token after six steps" is enough to keep several catoms moving at once while staying safe, and the same rules turn the chain around at the grid border, so the locomotion is endless.